
Learning to Push: Imitation Learning and RL on PushT

Department of Mechanical Engineering
2026.10.07 (Wed)
Joo Yong Sim

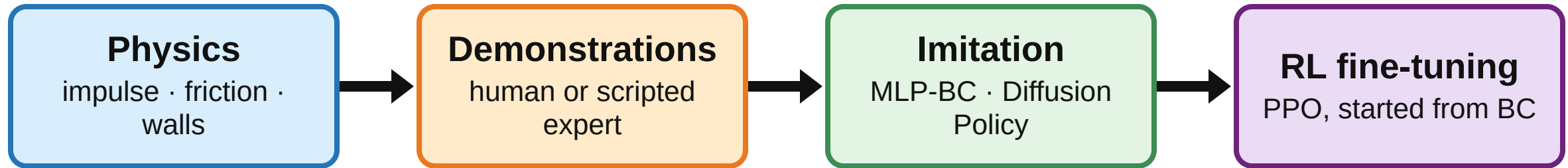


Hands-on tutorial: [pusht_imitation.py](#) · [pusht_rl.py](#) · [pusht_rl_realtime.py](#) · [pusht_rl_eval.py](#)

Lecture Goals: From Physics to Policy



The PushT simulator from the dynamics lecture becomes a training ground



Three questions answered with experiments in this lecture

Q1 — Data : how many demonstrations do we need to collect?

Q2 — Model : which network should learn from them — a regressor or a generative model?

Q3 — Beyond imitation : can reinforcement learning do better, from scratch or starting from the imitation policy?

Pipeline : demonstrations $D = \{(o_t, a_t)\}$ → dataset → policy $\pi_\phi(\mathbf{a} \mid o)$ → closed-loop success rate on unseen tasks

The PushT Task: Random Start, Random Goal



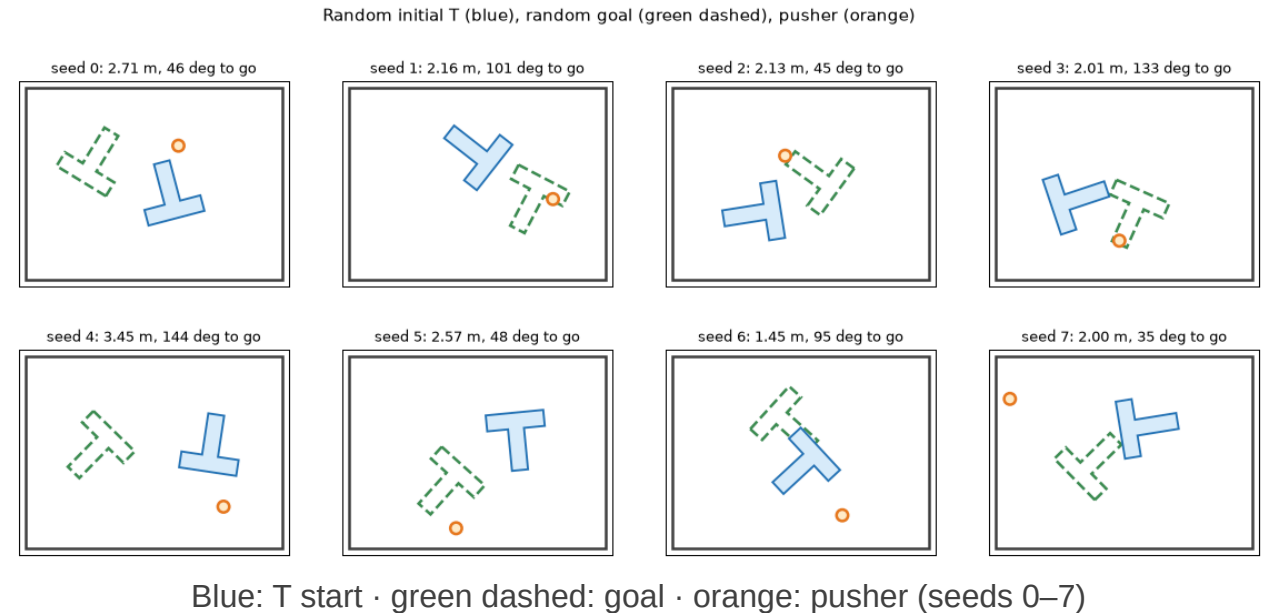
One seed fixes the initial T pose, the goal pose and the pusher position

What	Distribution
T start pose	$x \sim U(-2, 2)$, $y \sim U(-1, 1)$, $\theta \sim U(-\pi, \pi)$
Goal pose	same; redrawn if too easy (< 1 m and $< 45^\circ$)
Pusher	uniform in the arena, ≥ 1.6 from the T

Success

$$\|\mathbf{x}_G - \mathbf{x}_g\| < 0.15 \text{ m and } |\theta - \theta_g| < 10^\circ$$

Seeds : demos use 100000+ (expert) and 200000+ (human); **evaluation uses 0, 1, 2, ...** — tasks never seen in training.



Control Loop and Demonstration Data



The policy moves the pusher's target at 10 Hz; the physics runs 20 substeps in between

State (8 numbers — what a human sees)

$$s_t = (\underbrace{x_P, y_P}_{\text{pusher}}, \underbrace{x_G, y_G, \theta}_T, \underbrace{x_g, y_g, \theta_g}_{\text{goal}})$$

Action

$$a_t = (x^*, y^*) = \text{pusher target}, \quad \Delta t = 0.1 \text{ s}$$

A PD controller pulls the pusher toward the target;
contact with the T is the impulse model of the dynamics
lecture.

One episode, saved to .npz

$$\tau = ((s_0, a_0), (s_1, a_1), \dots, (s_{T-1}, a_{T-1}))$$

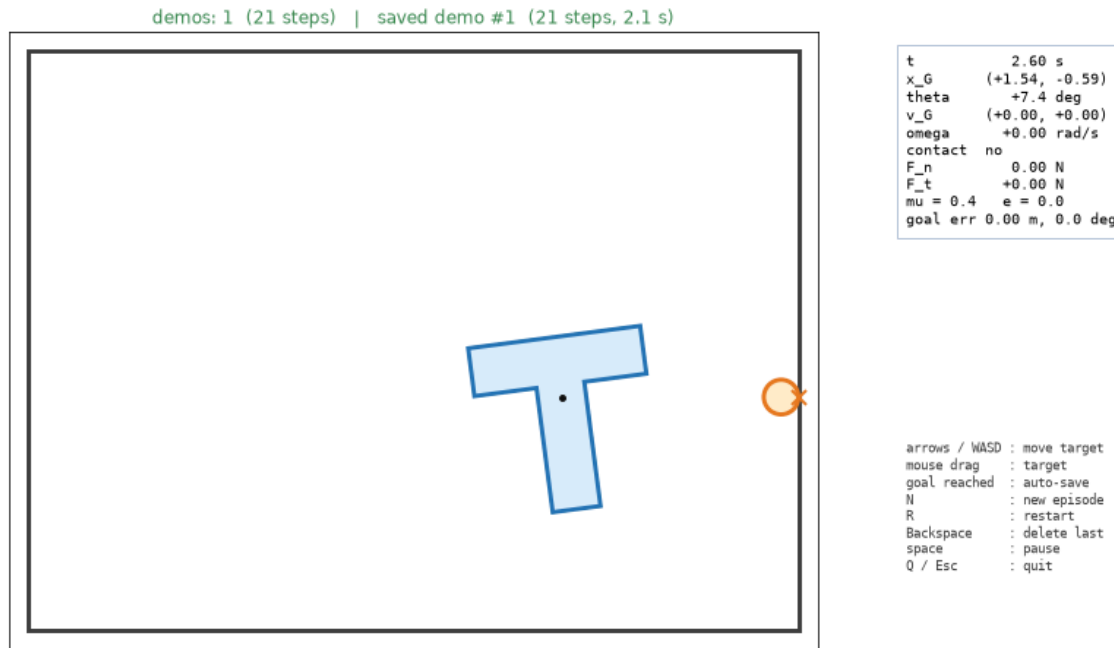
Array	Shape	Meaning
states	(N, 8)	s_t
actions	(N, 2)	$a_t = \text{target}$
episode_ends	(E,)	where each episode ends
seeds	(E,)	re-creates the start

Only successful episodes are saved : imitating a failure teaches failure (filtered behavior cloning). Same format as the Diffusion Policy / LeRobot PushT data.

Collecting Human Demonstrations



`python pusht_imitation.py collect` — the keyboard simulator with a recorder added



Recorder window: arrows / WASD or mouse drag move the target; reaching the goal auto-saves

Three rules for clean data

Fixed 10 Hz : the screen draws at 20 fps, but (s_t, a_t) is recorded every 0.1 s — one human step = one `env.step()`.

Idle start trimmed : time spent thinking before the first move is not recorded, so the policy never learns to stand still.

Physics fixed : friction / restitution switches are off — same state, same consequence.

Keys : N new task · R restart · Backspace delete last · Space pause

Cost : one human demo takes 15–30 s, so 200 demos take **1.5–2 hours** → we also need an automatic demonstrator.

Scripted Expert: Impulse–Momentum as a Policy



A "virtual human" that chooses where to push with the lecture's impulse equations

(1) Where to push — unit impulse $\mathbf{n}_k$ at candidate C_k

$$\Delta \mathbf{v}_G = \frac{\mathbf{n}_k}{m}, \quad \Delta \omega = \frac{\mathbf{r}_k \times \mathbf{n}_k}{I}$$

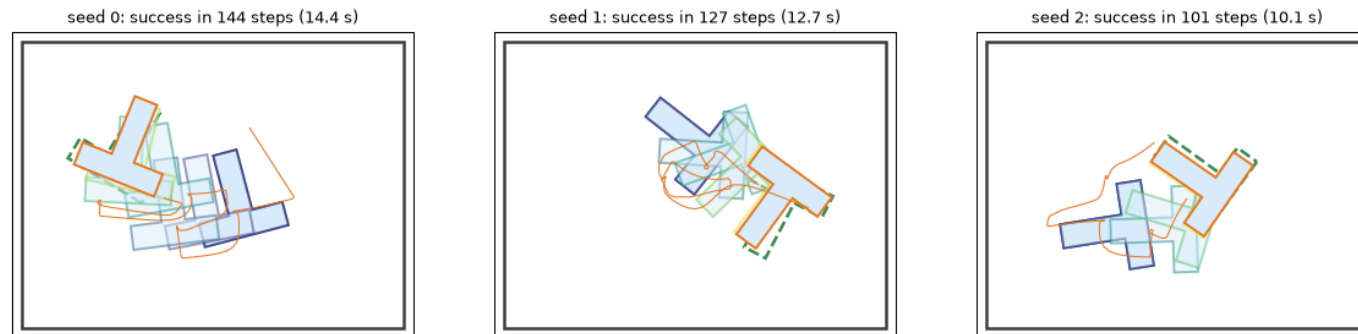
$$k^* = \arg \max_k \frac{\mathbf{c}_k \cdot \mathbf{d}}{\|\mathbf{c}_k\| \|\mathbf{d}\|}$$

Candidates : 8 edges $\times$ 3 points; $\mathbf{d}$ = desired $(\Delta x, \Delta y, L \Delta \theta)$, $\mathbf{c}_k$ = its effect.

(2) **Approach** : go to a standoff point; orbit around the T if the straight path is blocked.

(3) **Push** : along $\mathbf{n}_k$, slowing in proportion to the remaining error (the T slides $\approx v/c_v$ after release).

Scripted expert (orange line = pusher path, colour = time)



Expert rollouts (orange: pusher path, colour: time) — success $\approx$ 88–91 % on random tasks

How Many Demonstrations? Reference Points



Published PushT and manipulation work uses 50–200 demonstrations per task

Source	Task	Demonstrations
Diffusion Policy (Chi et al., 2023)	PushT, fixed goal , random start	200 human ($\approx$ 25k steps)
LeRobot pusht dataset	same task	206 episodes, 25,650 frames
ACT / ALOHA (Zhao et al., 2023)	real bimanual robot	50 per task

Suggested plan for human demonstrations (15–30 s each)

Stage	Demos	Time	Purpose
Pipeline check	10–20	10 min	save / load / train runs, loss goes down
First results	50–100	30–60 min	successes start; study the failures
Usable policy	$\geq$ 200	1.5–2 h	Diffusion Policy paper scale

More important than the count : spread-out initial conditions, **a consistent strategy**, and discarded failures. Our goal is random too — harder than the original PushT.

Goal Frame: Relative Motion Removes the Random Goal



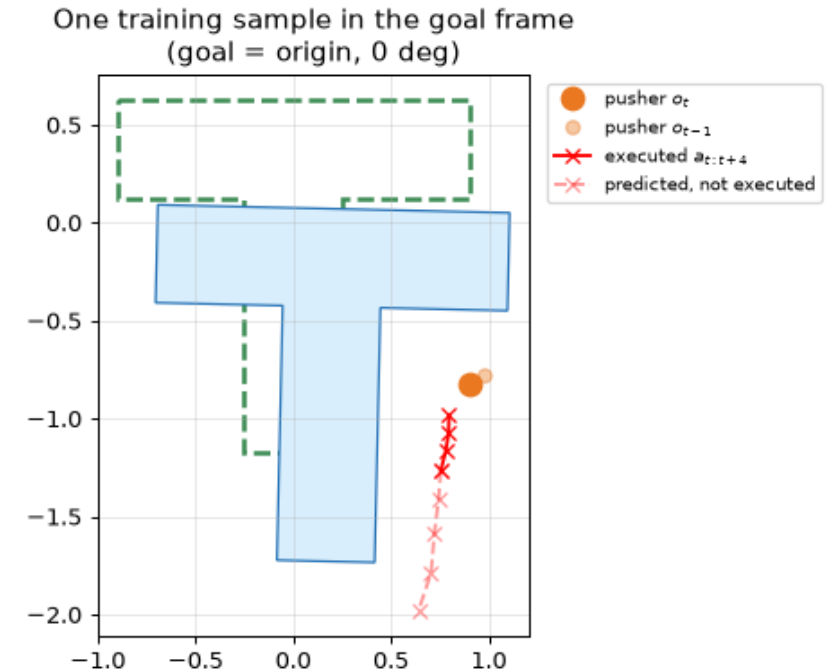
Ride along with the goal — the same idea as relative motion in rigid-body kinematics

$$\tilde{\mathbf{p}} = R(\theta_g)^\top (\mathbf{p} - \mathbf{x}_g), \quad \tilde{\theta} = \theta - \theta_g$$

$$\mathbf{o}_t = (\tilde{x}_P, \tilde{y}_P, \tilde{x}_G, \tilde{y}_G, \cos \tilde{\theta}, \sin \tilde{\theta})$$

$$\tilde{\mathbf{a}}_{t+i} = R(\theta_g)^\top (\mathbf{a}_{t+i} - \mathbf{x}_P(t))$$

Action representation (goal frame)	MLP-BC	Diffusion
absolute target position	26 %	20 %
offset from the current pusher	36 %	46 %



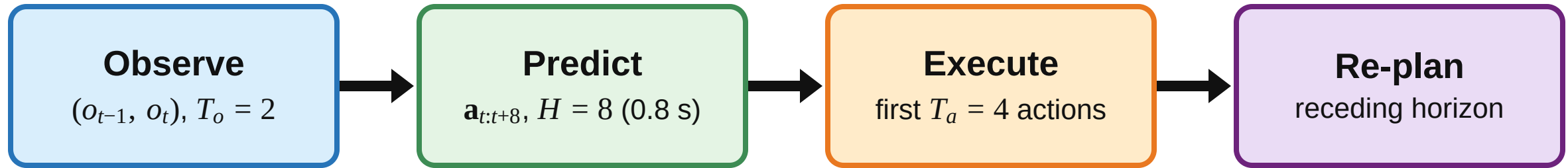
One training sample: goal at the origin, 0°

Without the goal frame : 0 % success for both models — a good representation saves data.

Observation History and Action Chunks



See two frames, plan eight actions, execute four, then re-plan



Two frames : the difference gives velocity — is the T still sliding?

Chunks : one-step predictions change every 0.1 s and jitter; a chunk commits to a motion for a moment.

Normalize : $z = (x - \mu)/\sigma$, statistics from the training set only.

Split by episode : neighbouring steps are nearly identical — a step-wise split leaks data into validation.

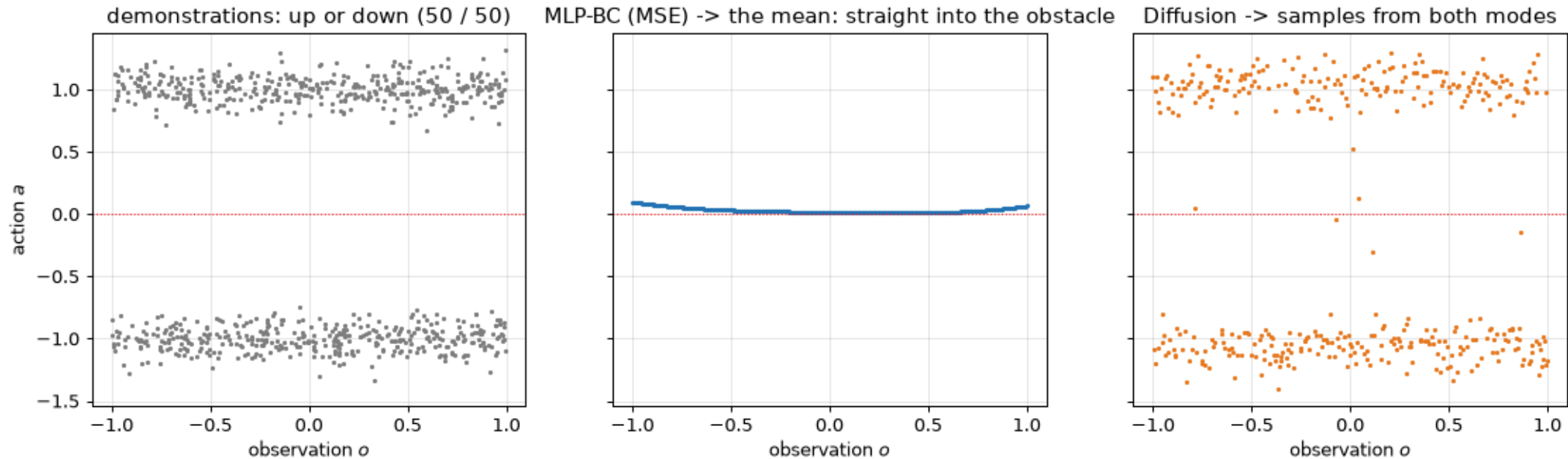
Trade-off : 4 steps run open-loop, so an unexpected slide is corrected only at the next re-plan.

Which Model? The Multimodality Problem



MSE learns the average action — the average of two good actions can be bad

$$\min_{\phi} \mathbb{E}_{(o,a) \sim D} \|\pi_{\phi}(o) - a\|^2 \implies \pi^*(o) = \mathbb{E}[a \mid o]$$



Toy problem: demonstrations avoid an obstacle up or down (50/50). MLP-BC learns $a \approx 0$ — straight into it; Diffusion samples both modes.

In PushT : "go around the left or the right side of the T?" — **MLP-BC hesitates in between**; human demos are even more multimodal.

Policy Model Candidates



Learn a distribution $p(a \mid o)$ instead of a single average action

Model	Output	Multimodal	Notes
MLP-BC (MSE)	one point	no (mean)	fastest, simplest — baseline
MLP-BC + action chunk	H points	no	temporally consistent
GMM / MDN	K Gaussians	partly	number of modes K fixed in advance
VQ-BeT	discrete code + offset	yes	Transformer
ACT (CVAE + Transformer)	chunk	yes	real robot, 50 demos
Diffusion Policy	chunk, sampled	yes	standard PushT baseline, stable training

Our choice : MLP-BC as the baseline, Diffusion Policy as the main model — **same MLP trunk** (3 layers, width 512, Mish).

So the only difference is : learning the mean vs. learning the distribution.



Diffusion Policy (DDPM) in Three Steps

Learn to remove noise from action chunks; at run time, denoise noise into actions

1. Forward process: mix noise into a clean chunk $\mathbf{a}^0$

$$\mathbf{a}^k = \sqrt{\bar{\alpha}_k} \mathbf{a}^0 + \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I), \quad k = 1, \dots, K$$

2. Training: predict the noise

$$L = \mathbb{E}_{k, \boldsymbol{\epsilon}} \left\| \epsilon_{\phi}(\mathbf{a}^k, k, o) - \boldsymbol{\epsilon} \right\|^2$$

3. Sampling: start from $\mathbf{a}^K \sim \mathcal{N}(\mathbf{0}, I)$, step $k = K, \dots, 1$

$$\hat{\mathbf{a}}^0 = \frac{\mathbf{a}^k - \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\epsilon}_{\phi}}{\sqrt{\bar{\alpha}_k}}, \quad \mathbf{a}^{k-1} = \frac{\sqrt{\bar{\alpha}_{k-1}} \beta_k}{1 - \bar{\alpha}_k} \hat{\mathbf{a}}^0 + \frac{\sqrt{\bar{\alpha}_k} (1 - \bar{\alpha}_{k-1})}{1 - \bar{\alpha}_k} \mathbf{a}^k + \sigma_k \mathbf{z}$$

Why it is multimodal : each run starts from different noise $\rightarrow$ a different sample $\rightarrow$ different modes of the demonstrations. Cosine schedule β_k , $K = 50$.



Training Settings Found by Experiment

Diffusion needs more capacity or more steps than MLP-BC before it pays off

Setting (180 expert demos, unseen tasks)

	MLP-BC	Diffusion
width 256, lr 10^{-3} , no EMA, 30k steps	32 %	8 %
width 256, lr 3×10^{-3} + EMA, 30k steps	36–46 %	18–42 %
width 256, lr 3×10^{-3} + EMA, 80k steps	—	64 %
width 512 , lr 3×10^{-3} + EMA, 30k steps (default)	38–59 %	54–63 %
width 512, lr 3×10^{-3} + EMA, 50k steps	—	69 %

$$\phi_{\text{EMA}} \leftarrow 0.999 \phi_{\text{EMA}} + 0.001 \phi$$

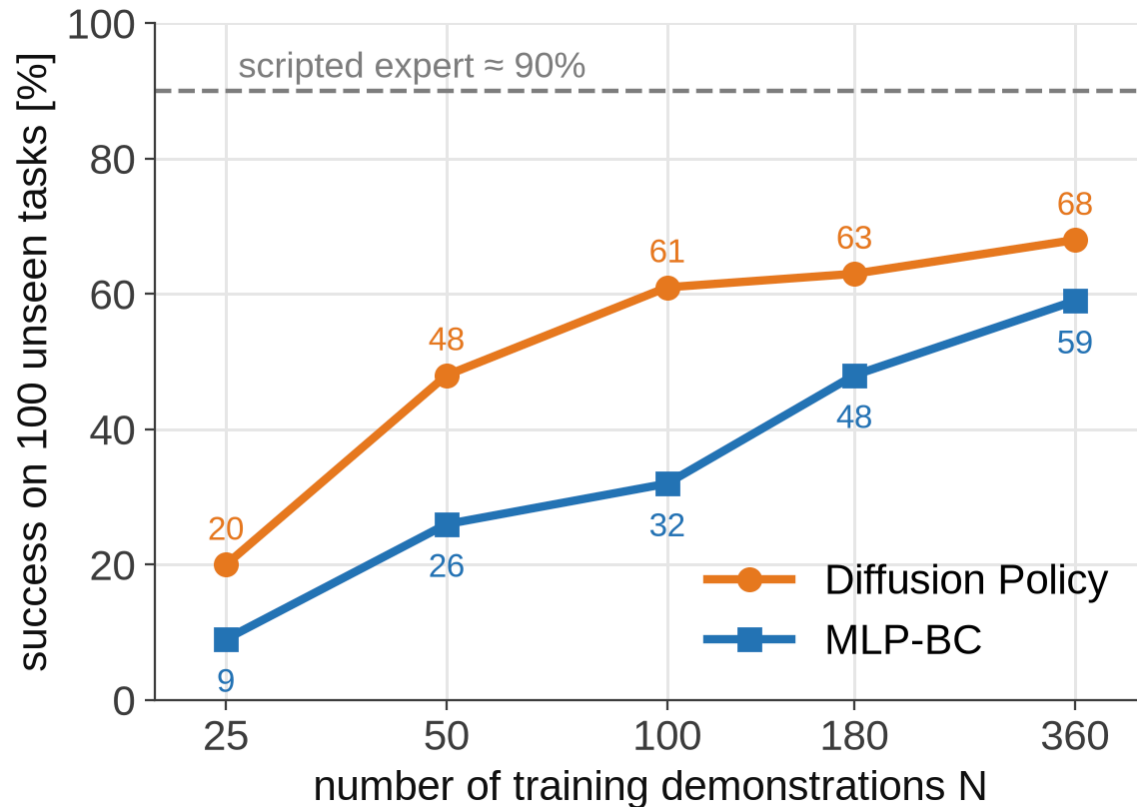
Batch 256, AdamW, cosine lr decay; about 1 min per model on GPU, 4 min on CPU.

Caution : re-running the same setting moves success by ± 10 %; validation loss does not predict success (0.116 $\rightarrow$ 0.132 while 62 % $\rightarrow$ 69 %).

How Many Demonstrations? Measured



Diffusion reaches most of its performance at about 100 demonstrations



Experiment

400 expert demos → train on the first N → width 512, 30k steps → **100 unseen tasks**, closed loop.

Reading

Diffusion is ahead at every N , most clearly with little data.

MLP-BC needs about 3× more demos for the same success.

Human demos : aim for **at least 100, ideally 200**.

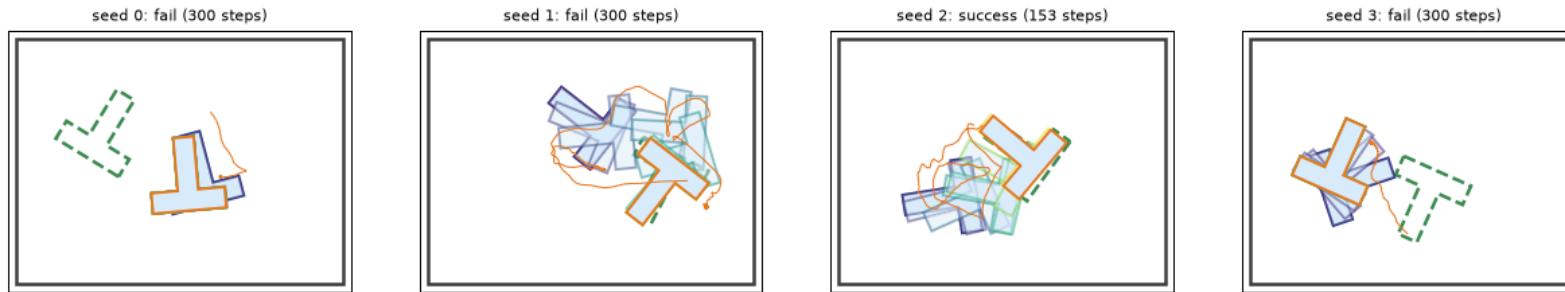
Reproduce : `tools/imitation_experiments/full/run_scale512.sh` (10 jobs, about 15 min)

Closed-Loop Rollouts and Failure Modes



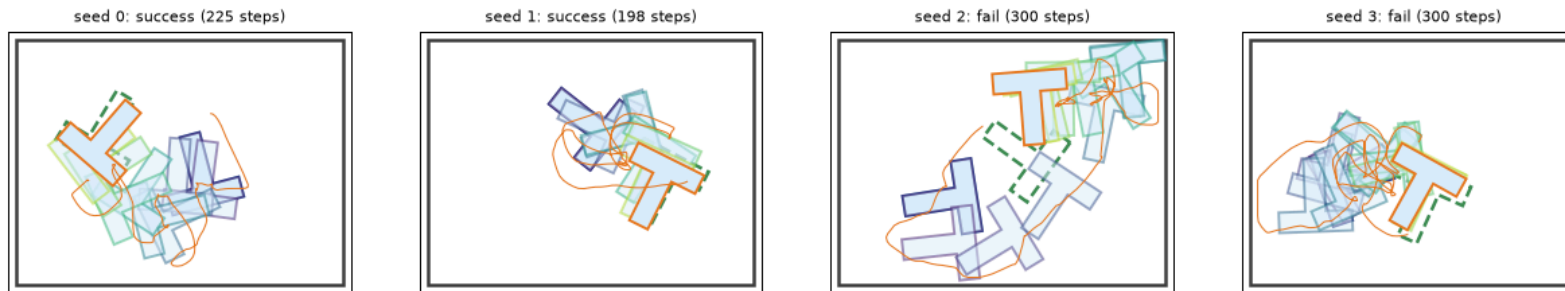
Both learned policies stay below the expert that taught them

MLP-BC rollouts on unseen seeds



MLP-BC rollouts on unseen seeds

Diffusion rollouts on unseen seeds



Diffusion Policy rollouts on the same seeds

MLP-BC : stalls or jitters beside the T — it averages between strategies.

Diffusion : commits to one strategy; fails at the final 0.15 m / 10° alignment or after one big mistake.

Both : **covariate shift** — their own errors lead to states never seen in the demos.

Default run (180 demos) : expert 88 % · MLP-BC 38–59 % · Diffusion 54–63 %. Remedy: DAgger — record human corrections.

From Imitation to Reinforcement Learning



RL needs no demonstrations — only a reward and a great deal of trial and error

$$s_t \xrightarrow{\pi_\phi(a|s)} a_t \xrightarrow{\text{physics}} s_{t+1}, r_t, \quad \max_{\phi} E\left[\sum_t \gamma^t r_t\right]$$

Experiment	Starting point	Result (50 unseen tasks, no exploration noise)
A. RL from scratch	random network	16.4M steps (2.5 h): 0 % success
B. BC → RL	MLP-BC policy	42 % → 74 % (3M) → 80 % (6M, 80 min)
Scripted expert	hand-written rules	88 %

Why A fails : success needs a long sequence — choose a contact, go around, push, align. Random motion almost never completes it, so the success bonus is **never observed** (the exploration problem).

Why B works : imitation already succeeds about half the time; RL only has to polish behaviour that works.

Reward Design



Turn "how far is the task from done" into one number, and reward every step of progress

Remaining cost (1 rad $\approx$ 0.3 m)

$$c(s) = \|\mathbf{x}_G - \mathbf{x}_g\| + 0.3 |\theta - \theta_g|$$

Reward per step

$$r_t = \underbrace{10(c(s_t) - c(s_{t+1}))}_{\text{progress}} - \underbrace{0.02 \max(\|\mathbf{x}_P - \mathbf{x}_G\| - 1, 0)}_{\text{far from T}} - \underbrace{0.01}_{\text{time}} + \underbrace{10 \cdot 1[\text{success}]}_{\text{bonus}}$$

Potential-based shaping : the progress term sums to $10(c(s_0) - c(s_T))$ — a signal every step, without changing the optimal policy.

Episode end : success is terminal; at 300 steps it is truncated and bootstrapped with $V(s_T)$.

Side effect : the time penalty makes RL **finish faster** than the expert, which has no such pressure.

PPO: Small, Clipped Policy Updates



Collect 8,192 steps, estimate advantages, update for 10 epochs — repeat

Advantage (GAE), $\gamma = 0.99$, $\lambda = 0.95$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad \hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}$$

Clipped objective, $\rho = \pi_\phi / \pi_{\text{old}}$

$$L = -\mathbb{E}[\min(\rho \hat{A}, \text{clip}(\rho, 0.8, 1.2) \hat{A})]$$

Gaussian policy $N(\mu_\phi(s), \sigma^2)$: sample while training, use the mean μ_ϕ for evaluation.

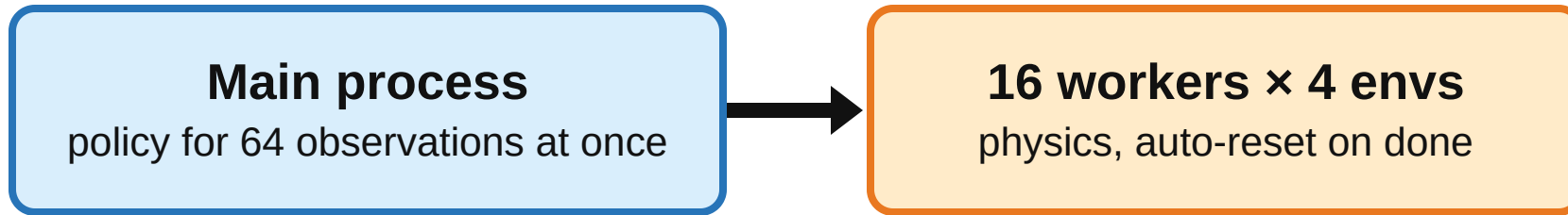
	A. scratch	B. BC $\rightarrow$ RL
Actor	$2 \times 256 \tanh$	the MLP-BC net
Initial σ	0.6	0.03 m
Actor lr	3×10^{-4}	3×10^{-5}
Critic	new	new, 10 warm-up updates

For B : clipping, a small learning rate and small noise **protect the working BC policy** from being destroyed early on.

Parallel Simulation



Pure-Python physics is slow, so 64 environments run at once on 16 CPU processes



Benchmark (idle CPU, 20 cores)

Steps per PPO iteration	$64 \times 128 = 8,192$
Rollout (physics 3.09 s)	3.79 s
PPO update	1.91 s (33 %)
Env steps / s	2,159 rollout · 1,437 total

Training wall-clock (from logs)

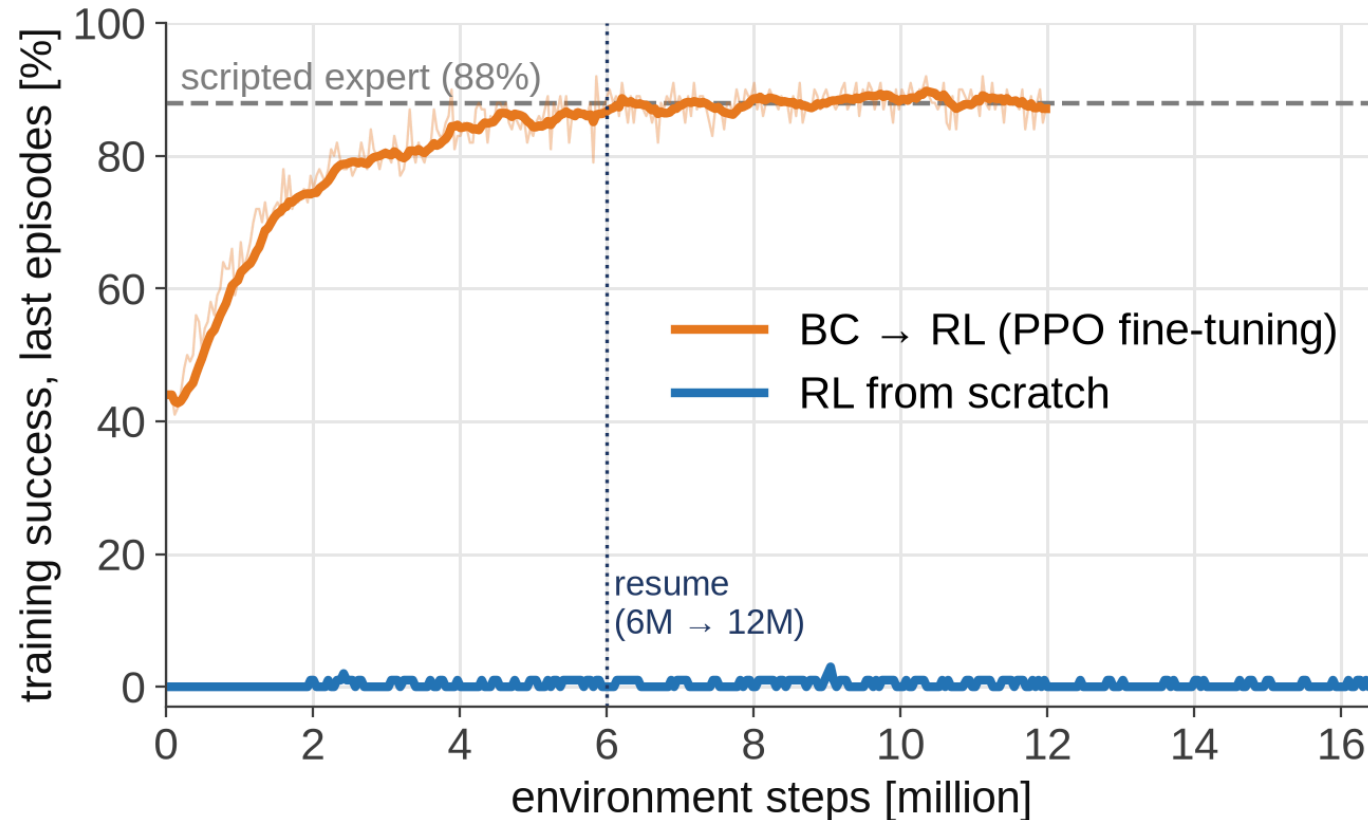
Run	Steps	Time
RL from scratch	16.4M	147 min
BC+PPO	0 → 6M	80 min
BC+PPO resumed	6 → 12M	70 min

Backend : CPU multiprocessing (fork + Pipe); the network also runs on the CPU. One environment alone would be **$\approx 64\times$ slower**.

RL from Scratch vs BC → RL



Starting from imitation turns an unsolvable exploration problem into fine-tuning



Training success with exploration noise (tutorial/checkpoints/rl/*/train_log*.txt)

From scratch : return improved $-14 \rightarrow -2.4$, the T moves toward the goal, but **0 % success**; exploration noise collapsed $0.6 \rightarrow 0.03$.

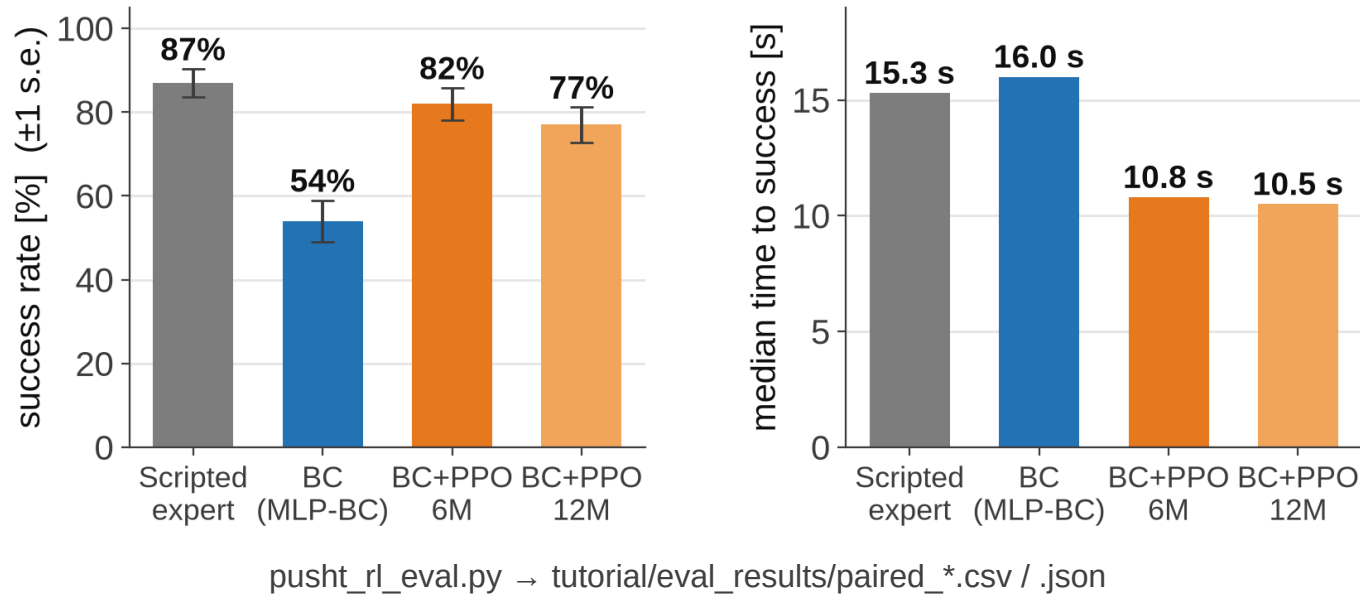
BC → RL : $0.44 \rightarrow 0.88$ in 6M steps, then a plateau at 0.87–0.90.

Lesson : demonstrations solve exploration; RL then optimizes what imitation cannot — the reward itself.

Paired Evaluation on Seeds 0–99



Same 100 unseen tasks, same start, 300 steps, deterministic actions for every policy



Scripted vs BC+PPO	6M	12M
both succeed	74	70
only scripted	13	17
only BC+PPO	8	7
both fail	5	6
McNemar p	0.38	0.064
median steps*	136 → 107	139 → 104

* on seeds both solved: scripted → BC+PPO

Result : success is statistically tied with the expert, and BC+PPO is about 3–4 s faster on the same tasks.

Why BC+PPO Stops Near the Expert



More training made the policy faster, not more successful

Measured

Within noise : at 82 %, one standard error is about ± 4 points with 100 tasks.

6M $\rightarrow$ **12M** : success 82 % $\rightarrow$ 77 %, mean time 12.0 s $\rightarrow$ 11.1 s.

Plateau : training success stayed at 0.87–0.90 with noise $\sigma \rightarrow 0.005$.

Best checkpoint : 6M, not the newest 12M.

Likely structural limits (not yet tested)

Less information : 12 inputs vs. the expert's full state and memory.

Conservative updates : lr 3×10^{-5} , clipping, small σ .

Reward $\neq$ success test : angle weighted 0.3; success needs 0.15 m and 10° .

One step at a time : BC drops 54 % $\rightarrow$ 34 % without chunks.

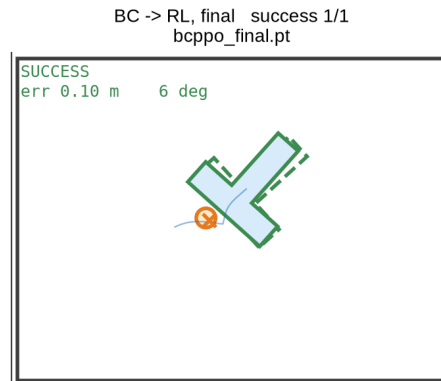
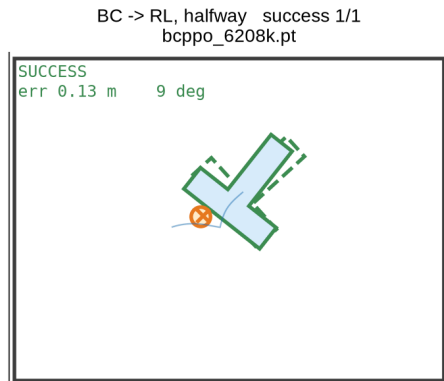
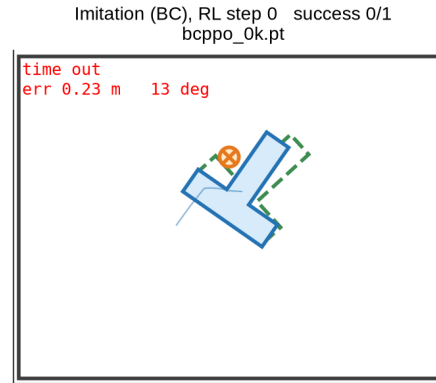
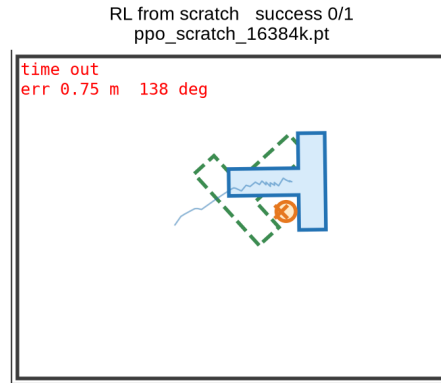
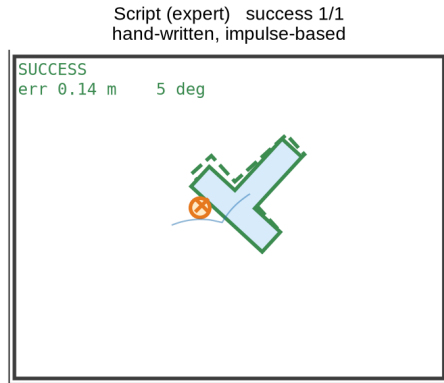
Not a ceiling : RL optimizes the reward, not imitation — align the reward with the success test to push further.

Real-Time Demo: Five Policies, One Task



`python pusht_rl_realtime.py` — every panel replays the same random task live

`pusht_rl_realtime.py` — all five policies on the same unseen task (seed 30)



Seed 30: script 8.7 s · BC → RL 6.4 / 6.2 s · BC and RL-from-scratch time out

Run

```
cd tutorial
python pusht_rl_realtime.py
python pusht_rl_realtime.py script
python pusht_rl.py eval
python pusht_rl_eval.py
```

Keys

Space pause · N next task · + / – speed ·
Q quit



Physics gives the data, imitation gives the start, RL gives the polish

- Imitation learning = supervised learning on $(o_t, \mathbf{a}_{t:t+H})$ from successful demonstrations
→ Goal frame (relative motion) and action offsets matter as much as the model
- Data: about 100 demos for Diffusion Policy; collect 100–200 human demos
- Model: MSE learns $E[a \mid o]$; **Diffusion learns the distribution** and handles multimodality
→ Measured: Diffusion 20–68 % vs MLP-BC 9–59 % (25–360 demos)
- RL from scratch: 0 % after 16M steps — the exploration problem
→ BC → PPO: 54 % → 82 % on seeds 0–99, tied with the expert (87 %) and 3–4 s faster



Exercises with the tutorial code

- **Human data** : collect 50, then 100 demos — does the Diffusion vs MLP-BC gap grow?
- **No goal frame** : train in world coordinates — how much success is lost?
- **Chunks** : vary the prediction / execution horizons (1, 4, 8, 16)
- **History** : one observation frame — which mistakes appear without velocity?
- **Dagger** : record your corrections in the viewer and retrain
- **Robustness** : evaluate with friction 0.2 instead of 0.4 — think domain randomization
- **RL** : put the 0.15 m / 10° test into the reward — does BC+PPO pass the expert?